API do projeto: o que é e onde está

May Artes 3D | explicacao simples com trechos reais do codigo

1. O que é uma API?

API é uma forma padronizada de um programa pedir dados ou serviços a outro programa. Pense nela como um atendente: recebe uma solicitação, encaminha para a parte certa do sistema e devolve uma resposta.

Neste projeto, a página do chatbot (frontend, no navegador) conversa por HTTP com o Django (backend, no servidor). O navegador faz uma requisição; a API executa a regra necessária e responde em JSON ou com um arquivo Excel.

NAVEGADOR	HTTP: GET / POST / PATCH	DJANGO API
home.js	home/urls.py -> home/views.py	models.py -> SQLite
	Resposta: JSON ou arquivo Excel	

2. Onde a API está no projeto?

- home/urls.py - registra os endereços da API e liga cada rota a uma view.
- home/views.py - implementa o que cada rota faz: lé dados, valida, consulta/salva no banco e monta a resposta.
- home/models.py - define as estruturas guardadas no SQLite: temas, perguntas, planilhas e registros.
- gerador_planilhas/urls.py - inclui as rotas do aplicativo home na raiz do site.
- templates/static/home/js/home.js - chama a API com fetch() e atualiza o chat com as respostas.

A API náo é um arquivo isolado: neste projeto ela é o conjunto formado por rotas, views, modelos e chamadas HTTP do navegador.

3. As rotas reais

Este trecho de home/urls.py mostra como os caminhos HTTP sáo associados ás funçóes Python que os atendem:

```
urlpatterns = [
    path("api/roteiro/", views.roteiro_chatbot,
         name="roteiro_chatbot"),
    path("api/planilhas/", views.listar_planilhas,
         name="listar_planilhas"),
    path("api/planilhas/criar/", views.criar_planilha,
         name="criar_planilha"),
    path("api/planilhas/<int:planilha_id>/",
         views.detalhe_planilha, name="detalhe_planilha"),
    path("api/planilhas/<int:planilha_id>/registros/",
         views.salvar_registro_planilha,
         name="adicionar_registro_planilha"),
    path("api/conversas/", views.criar_conversa,
         name="criar_conversa"),
]
```

Exemplo: GET /api/roteiro/ consulta o roteiro; POST /api/planilhas/criar/ envia dados para criar uma planilha. O trecho <int:planilha_id> representa um numero inteiro incluido no endereco.

4. O que uma view faz

roteiro_chatbot, em home/views.py, consulta os temas ativos e suas perguntas no banco e monta uma resposta JSON para o navegador:

```
@require_GET
def roteiro_chatbot(request):
    temas = TemaBot.objects.filter(ativo=True).prefetch_related("perguntas")
    mensagens = dict(MensagemBot.objects.values_list("chave", "texto"))
    roteiro = [
        {"codigo": tema.codigo, "nome": tema.nome,
         "perguntas": [
             {"campo": p.campo, "pergunta": p.pergunta,
              "tipo": p.tipo, "obrigatoria": p.obrigatoria,
              "opcoes": p.opcoes}
             for p in tema.perguntas.all()
         ]}
        for tema in temas
    ]
    return JsonResponse({"mensagens": mensagens, "temas": roteiro})
```

@require_GET restringe a view a requisicoes GET. JsonResponse transforma o dicionario Python em JSON, que o JavaScript consegue ler.

5. Exemplo: criar uma planilha

O navegador envia tema, nome e respostas. A view criar_planilha valida o JSON, verifica o tema e as respostas e grava a planilha e o primeiro registro dentro de uma transacao:

```
@require_POST
def criar_planilha(request):
    dados, erro_json = ler_json(request)
    if erro_json:
        return erro_json
    codigo_tema = dados.get("tema")
    nome = dados.get("nome")
    respostas = dados.get("respostas")
    # valida os campos e confirma que o tema esta ativo
    respostas_validadas, erro = validar_respostas(tema, respostas)
    if erro:
        return JsonResponse({"erro": erro}, status=400)
    with transaction.atomic():
        planilha = Planilha.objects.create(tema=tema, nome=nome.strip())
        registro = RegistroConversa.objects.create(
            tema=tema, planilha=planilha,
            respostas=respostas_validadas,
        )
    return resposta_salva(planilha, registro,
                          "planilha_criada", status=201)
```

Exemplo de requisicao

```
POST /api/planilhas/criar/
Content-Type: application/json

{
  "tema": "evento",
  "nome": "Feira de novembro",
  "respostas": {
    "nome_evento": "Feira Geek",
    "data_evento": "2026-11-04",
    "local": "Centro cultural",
    "status": "Planejado"
  }
}
```

Resposta de sucesso (resumo)

```
HTTP 201 Created
{
  "mensagem": "Planilha criada...",
  "planilha": {"id": 12, "nome": "Feira de novembro"},
  "registro": {"id": 31, "respostas": { ... }},
  "download_url": "/planilhas/arquivo/12/baixar/"
}
```

HTTP 201 significa que um recurso foi criado. Se os dados forem invalidos, a API responde com HTTP 400 e um JSON como {"erro": "..."}; recurso inexistente retorna HTTP 404.

6. Como o navegador chama a API

requestJson(), em templates/static/home/js/home.js, centraliza chamadas JSON. Para POST/PATCH envia Content-Type e o token CSRF; em seguida interpreta a resposta e transforma erros HTTP em mensagens.

```
async function requestJson(url, method = "GET", data = null) {
    const options = { method, headers: {} };
    if (data !== null) {
        options.headers["Content-Type"] = "application/json";
        options.headers["X-CSRFToken"] = csrfToken;
        options.body = JSON.stringify(data);
    }
    const response = await fetch(url, options);
    const result = await response.json();
    if (!response.ok) {
        throw new Error(result.erro || script.mensagens.erro_salvamento);
    }
    return result;
}
```

No fluxo de salvamento, saveEntry() escolhe a rota: criar uma planilha usa config.criar_planilha_url; adicionar registro usa /api/planilhas/{id}/registros/ com POST; editar registro acrescenta /{registro_id}/ e usa PATCH.

7. Outros recursos da API

- GET /api/planilhas/ e GET /api/planilhas/{id}/: listam planilhas ou carregam seus dados para edicao.
- POST /api/planilhas/externas/editar/: recebe um arquivo XLSX em multipart/form-data; primeiro devolve uma previa JSON e, com alteracoes, devolve uma copia editada.
- GET /planilhas/arquivo/{id}/baixar/ e GET /planilhas/{codigo}/baixar/: geram e devolvem arquivos Excel.

Em resumo: a rota identifica a operacao, a view executa a logica, os models acessam o banco e a resposta volta ao frontend. Para consultar o codigo completo, abra home/urls.py, home/views.py e templates/static/home/js/home.js.